

Absolutely. Since you've already covered **LLD (Low-Level Design)**, the next step is to learn **HLD (High-Level Design)** in an interview-oriented way.

For SDE interviews, don't think of HLD as "drawing complicated architecture diagrams." The real goal is:

Given a large-scale problem, design a system that is scalable, reliable, available, maintainable, and cost-effective.

1. LLD vs HLD

You can think of it like this:

LLD	HLD
Classes and objects	Services and components
Interfaces	APIs
Design patterns	Architecture patterns
Methods/functions	Data flow
SOLID	Scalability & reliability
Class diagrams	Architecture diagrams
In-memory data structures	Databases, caches, queues
Usually one service/module	Multiple distributed services
"How should this class work?"	"How should the whole system work?"

Example: Food Delivery System

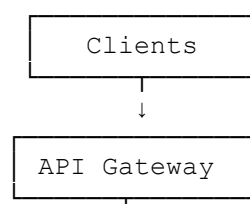
LLD asks:

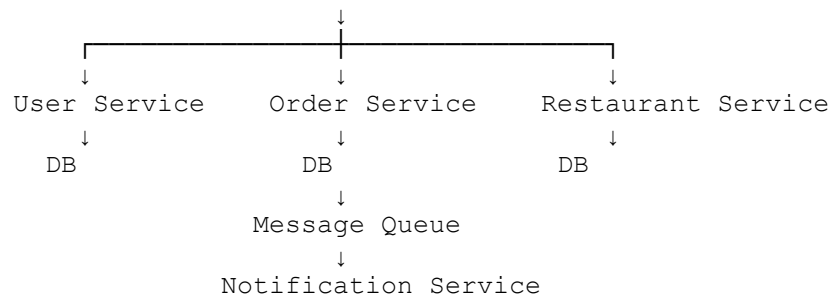
```
User
├── placeOrder()
└── cancelOrder()

Order
├── addItem()
└── calculatePrice()

Restaurant
└── acceptOrder()
```

HLD asks:





HLD is about **how the entire system communicates and scales**.

2. What exactly should you learn in HLD?

I recommend learning HLD in this order:

Phase 1 — Foundations

1. Client-server architecture
2. HTTP/HTTPS
3. REST APIs
4. DNS
5. IP addresses
6. TCP/UDP
7. Load balancers
8. Reverse proxy
9. API Gateway

Phase 2 — Databases

10. SQL databases
11. NoSQL databases
12. Indexing
13. Transactions
14. ACID
15. CAP theorem
16. Database replication
17. Sharding
18. Partitioning
19. Read replicas
20. Consistency

Phase 3 — Performance

21. Caching
22. Redis
23. CDN

- 24. Cache eviction
- 25. Cache invalidation
- 26. Connection pooling

Phase 4 — Distributed Systems

- 27. Message queues
- 28. Kafka
- 29. RabbitMQ
- 30. Pub/Sub
- 31. Event-driven architecture
- 32. Synchronous vs asynchronous communication
- 33. Idempotency
- 34. Distributed locks
- 35. Race conditions

Phase 5 — Reliability

- 36. Fault tolerance
- 37. Replication
- 38. Failover
- 39. Retry
- 40. Timeout
- 41. Circuit breaker
- 42. Rate limiting
- 43. Health checks
- 44. Disaster recovery
- 45. Backups

Phase 6 — Advanced

- 46. Microservices
- 47. Monolith
- 48. Service discovery
- 49. Containerization
- 50. Docker
- 51. Kubernetes basics
- 52. Observability
- 53. Logging
- 54. Metrics
- 55. Tracing

3. The most important HLD concepts

If you're preparing for interviews, these are the **core concepts you absolutely need**.

3.1 Client-Server Architecture

Basic architecture:

```
Client
  ↓
Server
  ↓
Database
```

Example:

When you open Instagram:

```
Mobile App
  ↓
Backend Server
  ↓
Database
```

The client sends a request:

```
GET /users/123
```

Server processes it and returns:

```
{
  "id": 123,
  "name": "Madhu"
}
```

4. Horizontal vs Vertical Scaling

This is extremely important.

Vertical Scaling

Increase the power of one machine.

Before:

```
Server
4 CPU
8 GB RAM
```

After:

```
Server
32 CPU
128 GB RAM
```

Advantages:

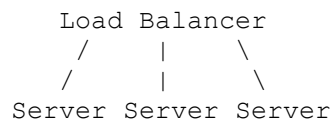
- Simple
- Easy to implement

Disadvantages:

- Hardware limit
- Expensive
- Single point of failure

Horizontal Scaling

Add more servers.



Instead of:

1 huge server

you have:

10 smaller servers

For large-scale systems, **horizontal scaling is generally preferred.**

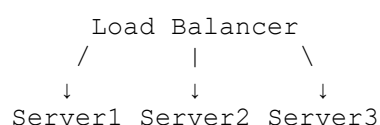
5. Load Balancer

Suppose you have:

10 million users

You shouldn't send every request to one server.

Instead:



The load balancer distributes traffic.

Common algorithms:

Round Robin

```
Request 1 → Server 1  
Request 2 → Server 2  
Request 3 → Server 3  
Request 4 → Server 1
```

Least Connections

Send request to the server with the fewest active connections.

IP Hash

Same client tends to go to the same server.

6. Stateless vs Stateful Servers

This is a very important interview concept.

Stateful

Server remembers client information.

```
Client A → Server 1
```

```
Server 1 remembers:  
User = Madhu  
Cart = [Laptop]
```

Problem:

If next request goes to Server 2:

```
Client A → Server 2
```

Server 2 doesn't know the session.

Stateless

Server doesn't store session information locally.

```
Client  
↓  
Load Balancer  
↓
```

Server 1 / Server 2 / Server 3

Session information can be stored in something like Redis or represented using tokens.

Therefore:

Stateless services are easier to horizontally scale.

7. Database

This is one of the biggest HLD topics.

Two major categories:

```
Database
├── SQL
└── NoSQL
```

SQL

Examples:

- PostgreSQL
- MySQL
- Oracle

Data:

Users

```
id | name | email
1  | Madhu | ...
2  | Rahul | ...
```

Good for:

- Transactions
 - Structured data
 - Relationships
 - Strong consistency
-

NoSQL

Examples:

- MongoDB

- Cassandra
- DynamoDB

Useful when:

- Huge scale
 - Flexible schema
 - High write/read throughput
 - Distributed architecture
-

8. Database Indexing

Suppose:

```
SELECT * FROM users
WHERE email = 'abc@gmail.com';
```

Without an index, database may scan:

```
1
2
3
4
5
...
10 million rows
```

With an index:

```
email index
  ↓
abc@gmail.com
  ↓
row 837291
```

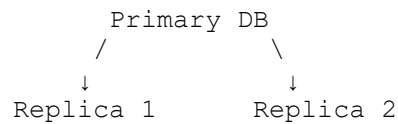
Much faster.

But indexes have a cost:

- Extra storage
 - Slower writes
 - Maintenance overhead
-

9. Database Replication

Suppose:



Primary handles writes.

Replicas can handle reads.

Example:

Write → Primary

Read → Replica 1

Read → Replica 2

This is called **read replication**.

Benefits:

- Better read scalability
- High availability
- Fault tolerance

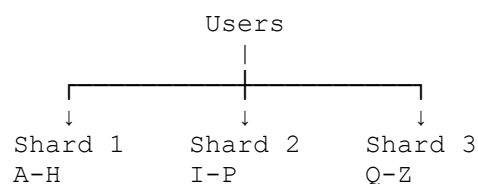
10. Database Sharding

Suppose your database has:

1 billion users

One database may become a bottleneck.

You can divide data:



This is **sharding**.

For example:

User ID 1-1M → Shard 1

User ID 1M-2M → Shard 2

User ID 2M-3M → Shard 3

Important interview question:

What happens if your shard key is bad?

You can get a **hot partition/shard**.

11. Caching

One of the most important HLD concepts.

Suppose:

```
Client
  ↓
Server
  ↓
Database
```

Every request hits DB.

Instead:

```
Client
  ↓
Server
  ↓
Cache
  ↓
Database
```

Example:

```
Request
  ↓
Redis
  ↓
Cache HIT → Return data
```

If cache miss:

```
Redis
  ↓
Database
  ↓
Redis
  ↓
Client
```

Popular cache:

Redis

12. Cache Eviction

Cache has limited memory.

Suppose cache is full.

Which item should be removed?

Common strategies:

LRU

Least Recently Used.

A B C D

If A hasn't been used recently:

Remove A

LFU

Least Frequently Used.

Remove the item used least often.

TTL

Data expires after a fixed time.

Example:

Cache user profile

TTL = 10 minutes

13. CDN

CDN = **C**ontent **D**elivery **N**etwork

Useful for static content:

Images
Videos
CSS
JavaScript

Instead of:

India User
↓
US Server

CDN can serve content from a nearby edge location.

India User
↓
India CDN Edge
↓
Content

This reduces latency.

14. Message Queue

This is another **must-learn HLD concept**.

Suppose an order is placed.

You don't necessarily want the order service to directly perform everything:

Order
↓
Payment
↓
Email
↓
SMS
↓
Analytics

Instead:

Order Service
↓
Message Queue
↓
├── Email
├── Payment
└── Analytics

Popular technologies:

- Kafka
- RabbitMQ
- AWS SQS

The queue allows **asynchronous processing**.

15. Kafka

Kafka is particularly important for interviews.

Basic idea:

```
Producer
  ↓
Kafka Topic
  ↓
Consumers
```

Example:

```
Order Service
  ↓
"order-created"
  ↓
Kafka
 /   \
↓     ↓
Payment Notification
Service Service
```

Important Kafka concepts:

- Producer
- Consumer
- Topic
- Partition
- Offset
- Consumer group
- Replication

16. Synchronous vs Asynchronous

Synchronous

A → B

A waits for B

Example:

```
User
  ↓
Payment Service
  ↓
Response
```

Asynchronous

A → Queue

A doesn't wait.

Example:

```
Order
↓
Kafka
↓
Notification
```

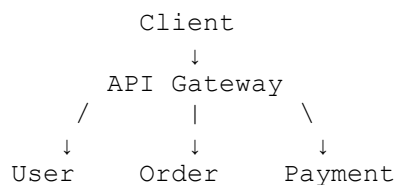
The order service doesn't have to wait for notification completion.

17. API Gateway

Instead of clients directly calling multiple services:

```
Client
├── User Service
├── Order Service
├── Payment Service
└── Product Service
```

Use:



API Gateway can handle:

- Authentication
 - Authorization
 - Rate limiting
 - Routing
 - Logging
 - Request transformation
-

18. Rate Limiting

Suppose one user sends:

100,000 requests/sec

Your system can become overloaded.

Rate limiter:

```
User
↓
Rate Limiter
↓
Allowed → Server
Blocked → 429
```

Example:

100 requests/minute/user

Algorithms worth knowing:

- Token bucket
- Leaky bucket
- Fixed window
- Sliding window

19. CAP Theorem

Very important for HLD interviews.

CAP stands for:

C — Consistency

Every user sees the latest data.

A — Availability

Every request receives a response.

P — Partition tolerance

System continues working despite network partition.

The important point:

In a distributed system, when a network partition occurs, you generally have to choose between strong consistency and availability.

You don't simply say:

Choose any 2 of 3

That oversimplifies CAP.

20. Consistency

Suppose:

User transfers ₹100

You don't want:

Account A = ₹1000
Account B = ₹1000

after the transfer if the system has inconsistent reads.

Types you'll encounter:

Strong consistency

Latest write is immediately visible.

Eventual consistency

Different replicas may temporarily disagree, but eventually converge.

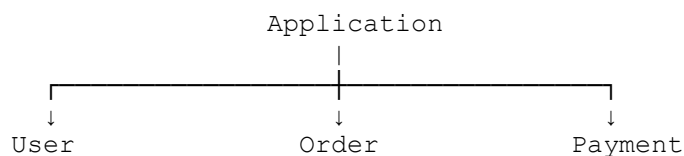
Example:

Social media likes can often tolerate eventual consistency.

Bank balance generally needs much stronger consistency guarantees.

21. Microservices

Instead of one giant application:



Each service can be independently deployed.

Advantages:

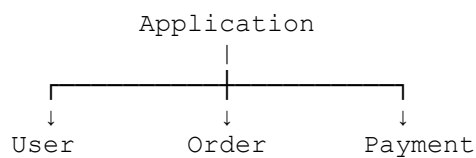
- Independent scaling
- Independent deployment
- Team ownership
- Fault isolation

Disadvantages:

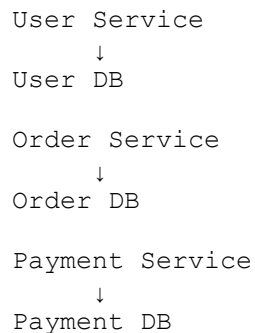
- Network complexity
- Distributed debugging
- Data consistency problems
- Deployment complexity

22. Monolith vs Microservices

Monolith



Microservices



Don't automatically choose microservices.

A good interview answer is:

"I would start with a modular monolith if the scale and organizational complexity don't justify microservices, and split services when independent scaling, deployment, or ownership becomes necessary."

That's a much stronger answer than simply saying "**microservices because it's scalable.**"

23. Reliability Concepts

You should know:

Retry

```
Request
↓
Failure
↓
Retry
```

But don't retry infinitely.

Use:

Exponential Backoff

Example:

```
1 sec
2 sec
4 sec
8 sec
```

Timeout

Don't allow a request to wait forever.

```
Request
↓
Service
↓
Timeout → Failure
```

Circuit Breaker

If a service is continuously failing:

```
Service A
↓
Service B ✕
```

Instead of continuously calling B:

```
Circuit OPEN
↓
Stop requests to B temporarily
```

This protects the rest of the system.

24. HLD Interview Framework

This is probably the **most important thing for you to learn**.

When interviewer says:

"Design YouTube."

Don't immediately start drawing boxes.

Follow this sequence.

Step 1 — Clarify Requirements

Ask:

Functional requirements

What should the system do?

For YouTube:

1. Upload video
2. Watch video
3. Search videos
4. Like video
5. Comment

Non-functional requirements

How should it behave?

High availability
Low latency
Scalable
Durable
Fault tolerant

25. Step 2 — Estimate Scale

Suppose interviewer says:

100 million users

Estimate:

DAU
Requests/sec
Storage
Bandwidth

For example:

100M users
10M DAU
5 requests/user/day

= 50M requests/day

Then:

50M / 86400
≈ 579 requests/sec

Add peak traffic:

~5-10× average

Maybe:

~3,000-6,000 RPS peak

The exact number isn't the point.

The interviewer wants to see **how you reason about scale**.

26. Step 3 — Define APIs

Example:

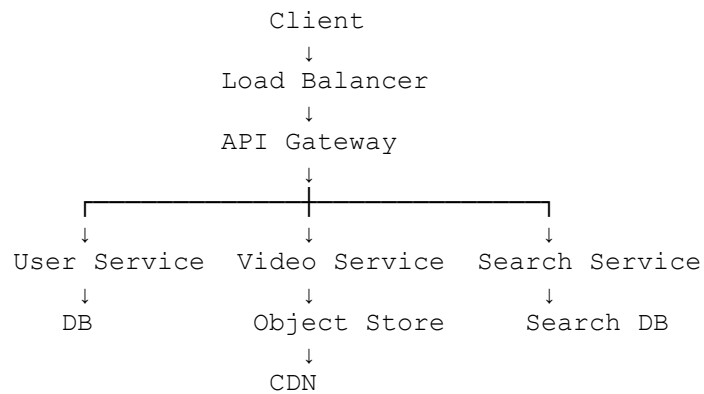
POST /videos
GET /videos/{id}
GET /videos/{id}/stream
POST /videos/{id}/like
POST /videos/{id}/comments

Define:

Input
Output
Authentication
Errors

27. Step 4 — Design High-Level Architecture

Then draw:



Now you're actually doing HLD.

28. Step 5 — Database Design

Ask:

What data?
How much data?
Read/write ratio?
Relationships?
Consistency?

For example:

Users
Videos
Comments
Likes
Subscriptions

Then decide:

SQL?
NoSQL?
Object storage?
Search engine?
Cache?

29. Step 6 — Identify Bottlenecks

Ask:

What happens when traffic becomes 10× larger?

Potential bottlenecks:

```
Database
↓
Cache

Server
↓
Load balancing

Storage
↓
CDN

Heavy processing
↓
Queue
```

This is where your HLD becomes strong.

30. Step 7 — Failure Handling

Ask:

What if this component goes down?

Example:

```
DB goes down
↓
Replica / Failover
Server goes down
↓
Load balancer routes elsewhere
Payment service goes down
↓
Retry + Queue + Circuit breaker
```

31. Step 8 — Deep Dive

After the architecture is ready, interviewer may say:

"How would you scale the database?"

Then discuss:

Read replicas
Caching
Sharding
Partitioning
Indexes

Or:

"How would you handle 1M video uploads?"

Discuss:

Object storage
Pre-signed URLs
Async processing
Message queues
Transcoding workers
CDN

32. HLD Problems You Should Practice

Don't randomly practice 50 systems.

Start with these:

Beginner

1. URL Shortener
2. Rate Limiter
3. Pastebin
4. File Storage System
5. Parking Lot at system level

Intermediate

6. WhatsApp
7. Chat System
8. Food Delivery
9. Ride Sharing
10. E-commerce
11. Notification System
12. News Feed

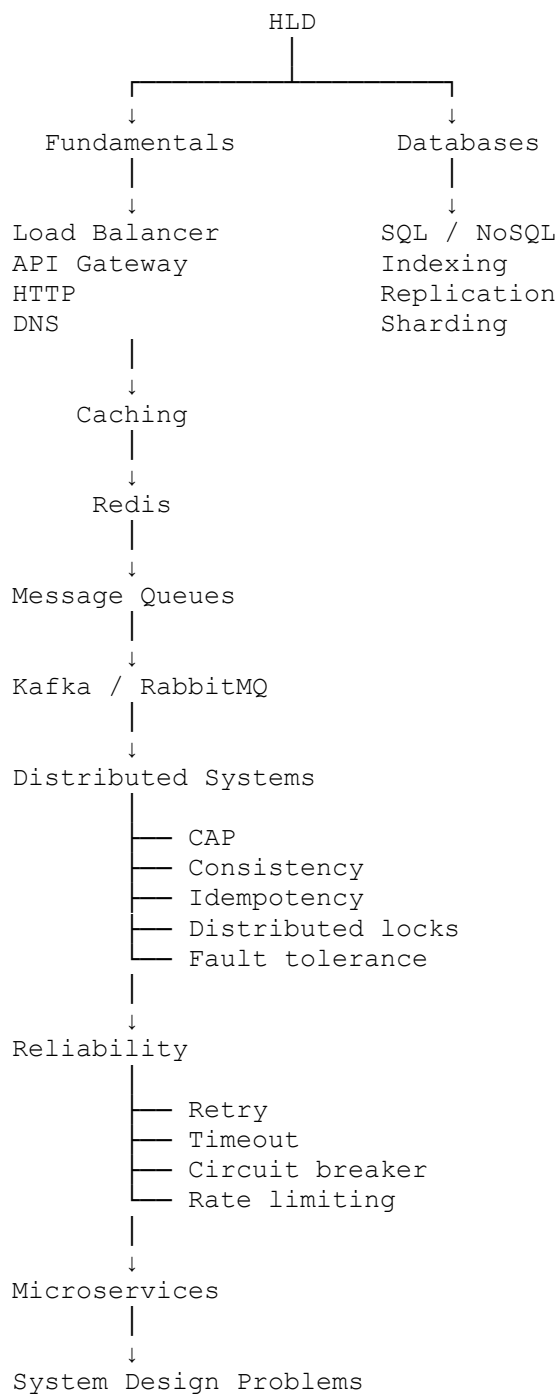
Advanced

13. YouTube
14. Netflix
15. Instagram
16. Uber

17. Google Drive
 18. Dropbox
 19. Ticket Booking
 20. Distributed Web Crawler
-

33. Your HLD Learning Roadmap

Since you've already done LLD, I'd recommend:



34. What you DON'T need initially

Since you're preparing for interviews, don't go too deep into infrastructure initially.

You **don't need to master**:

- Kubernetes internals
- Docker internals
- Raft implementation
- Paxos implementation
- Linux kernel internals
- AWS internals
- Kubernetes scheduler internals

Learn the **concept and when/why you'd use it** first.

For example:

What is Kubernetes?

You should know:

It orchestrates containers, handles deployment, scaling, service discovery, and self-healing.

You don't initially need to know Kubernetes source code.

35. The HLD mindset

The biggest transition from LLD to HLD is this:

LLD thinking

"How do I implement this class?"

HLD thinking

"What happens when 10 million users use this system simultaneously?"

Then ask:

Can it scale?

↓

Can it survive failures?

↓

Can it handle traffic spikes?

↓

Where is the bottleneck?

↓

What happens if DB fails?

↓

How do I reduce latency?

↓

How do I maintain consistency?

↓

How much will it cost?

That's **HLD thinking**.

What I recommend for you

Since you've already covered LLD, **don't start HLD by memorizing complete designs for Uber, Netflix, YouTube, etc.**

First master these **12 building blocks**:

1. Load Balancer
2. API Gateway
3. Caching / Redis
4. SQL vs NoSQL
5. Indexing
6. Replication
7. Sharding
8. CDN
9. Message Queue
10. Kafka
11. CAP + Consistency
12. Rate Limiting + Reliability

Then practice systems using those blocks.

A very good next sequence for you would be:

Load Balancer → Caching → Database Scaling → Sharding → Message Queues/Kafka → CAP → Rate Limiting → Microservices → URL Shortener → WhatsApp → YouTube → Uber.

If you follow that sequence, you'll build HLD understanding rather than just memorizing diagrams.